

“TalentFlow”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

KASULA YASHAS : 2520030401

GNANESHWAR REDDY : 2520030552

CHARAN : 2520030321

Under the Guidance of

Dr. V.Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled “**TalentFlow**” is a record of Bonafide work of **KASULA YASHAS - 2520030401, GNANESHWAR REDDY - 2520030552, CHARAN – 2520030321** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

KASULA YASHAS – 2520030401

GNANESHWAR REDDY – 2520030552

CHARAN – 2520030321

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled “**TalentFlow**” is a bonafide work done and submitted **KASULA YASHAS - 2520030401, GNANESHWAR REDDY - 2520030552, CHARAN – 2520030321** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27**.

Signature of the Guide
(Dr. V.Sireesha)

Signature of the HOD

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for her valuable guidance, continuous support, and constructive suggestions throughout the development of our project, “TalentFlow – Resume Search and Candidate Screening System.” Her guidance helped us understand the project requirements, apply appropriate Data Structures and Algorithms, and systematically develop and test the proposed system.

We would also like to express our sincere thanks to Dr. P. Venkateswara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the necessary academic support, resources, and encouragement to complete this project successfully. We are grateful to the Department of Computer Science and Engineering for providing us with an environment that enabled us to apply our technical knowledge to a practical problem.

Finally, we would like to thank everyone who directly or indirectly supported us during the development and documentation of this project. The completion of this work has helped us strengthen our understanding of Data Structures and Algorithms, Java, Spring Boot, React, resume processing, candidate matching, and system development.

KASULA YASHAS – 2520030401
GNANESHWAR REDDY – 2520030552
CHARAN – 2520030321

Abstract

TalentFlow is a Resume Search and Candidate Screening System developed to simplify the process of searching, analyzing, and screening candidates based on information available in their resumes. The system addresses the difficulty of manually reviewing large numbers of resumes by providing a structured platform for resume processing, candidate search, matching, and screening. It is designed to apply Data Structures and Algorithms to practical recruitment-related tasks.

The system provides functionality for resume uploading and parsing, candidate search, resume matching, candidate optimization, and interview scheduling. The backend is implemented using Java and Spring Boot, with Apache PDFBox used for extracting information from PDF resumes. The project incorporates several algorithmic techniques, including Aho-Corasick, KMP, Rabin-Karp, Edit Distance, Z-Function, Suffix Array, Resume Similarity, Bipartite Matching, Candidate Optimization, and Scalable Resume Processing. These algorithms are integrated into the backend to support efficient text searching, similarity analysis, candidate matching, and processing of resume data.

The system follows a full-stack architecture with a React-based frontend and a Spring Boot backend connected through REST APIs. The frontend provides the user interface for interacting with the system, while the backend manages resume processing, candidate information, search operations, matching services, and scheduling functionality. By integrating these components into a single system, TalentFlow demonstrates how Data Structures and Algorithms can be applied to develop a practical candidate screening and resume search application.

List of Figures

Figure No.	List of Figures	Page No.
3.1	Use Case Diagram of TalentFlow	26
3.2	Data Flow Diagram of TalentFlow	27
3.11	System Architecture of TalentFlow	26
5.16	Overall System Workflow	37
6.1	Login Screen	42
6.2	Dashboard	43
6.3	Resume Upload	44
6.4	Candidate Search	45
6.5	Ranked Search Results	45
6.6	Candidate Details	46

Table of Contents

S.No	Contents	Page No.
	Acknowledgement Abstract Table of Contents List of Figures	
1	Introduction	11 -18
	1.1 Introduction 1.2 Background of the Project 1.3 Problem Statement 1.4 Motivation 1.5 Scope of the Project 1.6 Objectives 1.7 Proposed System 1.8 Advantages of the Proposed System 1.9 Applications 1.10 Organization of the Report	
2	System Requirements and Existing System	19 - 21
	2.1 Introduction 2.2 Existing Resume Screening Systems 2.3 Resume Search Techniques 2.4 String Matching Algorithms 2.5 Resume Similarity Techniques 2.6 Candidate Matching and Ranking 2.7 Existing System Limitations 2.8 Research Gap 2.9 Summary	
3	System Design	22 - 27
	3.1 Introduction	

	3.2 Functional Requirements 3.3 Non-Functional Requirements 3.4 Hardware Requirements 3.5 Software Requirements 3.6 Technology Stack 3.7 Frontend Technologies 3.8 Backend Technologies 3.9 Database / Data Management 3.10 Development Tools 3.11 System Architecture 3.12 System Workflow 3.13 Use Case Diagram 3.14 Data Flow Diagram 3.15 Summary	
4	DSA Algorithms and Implementation	28 - 31
	4.1 Introduction 4.2 Aho-Corasick Algorithm 4.3 Knuth-Morris-Pratt (KMP) Algorithm 4.4 Rabin-Karp Algorithm 4.5 Z-Function Algorithm 4.6 Edit Distance Algorithm 4.7 Suffix Array 4.8 Resume Similarity 4.9 Bipartite Matching 4.10 Candidate Optimization and Ranking 4.11 Scalable Resume Processing 4.12 Algorithm Integration in TalentFlow 4.13 Complexity Analysis 4.14 Summary	

5	Implementation	32 - 37
	5.1 Introduction 5.2 Implementation Architecture 5.3 Resume Upload Module 5.4 Resume Parsing and Information Extraction 5.5 Candidate Information Management 5.6 Candidate Search Module 5.7 Resume Similarity and Text Comparison 5.8 Duplicate Detection 5.9 Candidate Matching and Optimization 5.10 Candidate Ranking 5.11 Interview Scheduling Module 5.12 REST API Implementation 5.13 React Frontend Implementation 5.14 Spring Boot Backend Implementation 5.15 Frontend-Backend Integration 5.16 Overall Implementation Flow 5.17 Implementation Summary	
6	Results, Testing and Analysis	38 - 48
	6.1 Introduction 6.2 System Execution Flow 6.3 Functional Testing 6.4 Resume Upload Testing 6.5 PDF Text Extraction Testing 6.6 Candidate Search Testing 6.7 Edit Distance Testing 6.8 Candidate Matching and Ranking Testing 6.9 Algorithm Testing 6.10 System Integration Testing	

	6.11 User Interface Results 6.12 Performance Analysis 6.13 Expected System Results 6.14 Test Result Summary 6.15 Analysis 6.16 Summary	
7	Challenges and Limitations	49 - 51
	7.1 Issues Faced During Development 7.2 Solutions Applied 7.3 Limitations of the System 7.4 Summary	
8	Conclusion and Future Work	52 - 55
	8.1 Conclusion 8.2 Future Work 8.3 Skills Learned During Development 8.4 Summary References	

1. INTRODUCTION

1.1 Overview

TalentFlow is a Resume Search and Candidate Screening System developed to simplify and improve the process of managing, searching, analyzing, and screening candidate resumes. In a conventional recruitment process, recruiters may need to manually examine a large number of resumes to identify candidates whose qualifications, skills, education, work experience, and certifications match the requirements of a job. This manual process can require significant time and effort and may make it difficult to compare candidates consistently.

TalentFlow provides a structured platform for uploading, processing, storing, searching, and comparing candidate resumes. The system extracts important information from resumes and makes the information available for efficient candidate search and screening. Recruiters can use keywords and filtering mechanisms to identify candidates based on relevant qualifications and skills.

The system is designed as a full-stack application with a React-based frontend and a Java Spring Boot backend. The backend handles resume processing, candidate information, search operations, matching services, and related system functionality, while the frontend provides an interface through which recruiters can interact with the application. The project abstract also identifies PDF resume processing and automated extraction of candidate information as important functions of the system.

A major focus of TalentFlow is the application of Data Structures and Algorithms to practical recruitment-related problems. The system incorporates algorithmic techniques for text searching, similarity analysis, candidate matching, and candidate optimization. This makes the project not only a recruitment application but also a practical demonstration of how Data Structures and Algorithms can be integrated into a real-world software system.

The overall purpose of TalentFlow is to reduce the effort involved in manual resume screening and provide recruiters with a more organized and efficient method of finding suitable candidates.

1.2 Background

Recruitment involves several activities, including collecting candidate resumes, examining qualifications, identifying relevant skills, comparing candidate profiles, and selecting suitable applicants for further evaluation. As the number of applications increases, manually performing these activities becomes increasingly difficult.

A resume generally contains different types of information, such as contact details, educational qualifications, technical skills, work experience, and certifications. Identifying relevant information from each resume manually can take considerable time. Furthermore, searching for particular skills or qualifications across a large collection of resumes can become inefficient when the information is not organized in a structured manner.

Resume screening systems are therefore useful for organizing candidate information and supporting recruiters during the initial stages of recruitment. TalentFlow addresses this requirement by providing facilities for resume uploading, resume parsing, candidate searching, matching, and ranking.

The project applies Data Structures and Algorithms to the processing and searching of resume information. String matching algorithms can be used to locate relevant terms efficiently, while similarity and matching techniques can assist in comparing candidate profiles with recruitment requirements. The abstract specifically identifies algorithms such as Aho-Corasick, KMP, Rabin-Karp, Edit Distance, Z-Function, Suffix Array, Resume Similarity, Bipartite Matching, Candidate Optimization, and Scalable Resume Processing as part of the system.

By combining these algorithmic techniques with a web-based application, TalentFlow demonstrates the practical application of DSA concepts to a recruitment problem.

1.3 Problem Statement

The traditional resume screening process often requires recruiters to manually inspect and compare a large number of resumes. This creates several difficulties when the number of candidates increases.

The major problem is the time required to identify candidates who satisfy the requirements of a particular job. Recruiters may need to search through individual resumes to find specific skills, educational qualifications, work experience, or certifications. Manual screening can also make it difficult to maintain a consistent method of comparing candidates.

Another challenge is the organization and retrieval of resume information. When resumes are stored without a structured search and processing mechanism, finding a particular candidate or identifying candidates with specific skills can become inefficient.

Therefore, there is a need for a system that can:

- Upload and process candidate resumes.
- Extract relevant information from resumes.
- Store candidate information in an organized manner.
- Search resumes using relevant keywords.
- Compare candidate profiles efficiently.
- Identify candidates whose profiles match job requirements.
- Support candidate ranking and screening.
- Reduce the time and manual effort required during initial recruitment.

TalentFlow is developed to address these problems by providing an integrated platform for resume processing, candidate search, matching, and screening.

1.4 Motivation

The primary motivation for developing TalentFlow is the increasing difficulty of manually processing and screening a large collection of candidate resumes. Recruiters need to identify suitable candidates efficiently while considering information such as technical skills, education, experience, and certifications.

Manual resume screening can become repetitive and time-consuming, particularly when many candidates apply for the same position. An organized system that can process resume information and provide efficient search and matching capabilities can help reduce this workload.

Another motivation for the project is to demonstrate the practical application of Data Structures and Algorithms. Instead of studying algorithms only as theoretical concepts, TalentFlow applies them to operations that occur in an actual software system.

String-searching algorithms can support efficient keyword searching, while similarity and matching algorithms can help compare candidate information. This provides a practical environment for understanding how algorithm selection can influence the efficiency of software applications.

The project therefore combines two objectives: developing a useful resume screening application and demonstrating the application of Data Structures and Algorithms to a real-world problem.

1.5 Scope of the Project

The scope of TalentFlow covers the processing, management, searching, and screening of candidate resumes. The system is intended to provide recruiters with a centralized platform through which candidate information can be processed and evaluated.

The major areas covered by the project include:

1. **Resume Upload**

The system provides functionality for uploading candidate resumes for processing.

2. **Resume Parsing**

Resume documents can be processed to extract relevant candidate information such as skills, education, work experience, certifications, and contact details.

3. **Candidate Management**

Extracted candidate information can be organized and accessed through the application.

4. **Resume Search**

Recruiters can search candidate information using relevant keywords and search criteria.

5. **Candidate Matching**

Candidate profiles can be compared with job-related requirements to identify relevant candidates.

6. **Resume Similarity**

Similarity techniques can be used to compare resume information and support candidate analysis.

7. **Candidate Optimization and Ranking**

The system provides mechanisms for evaluating candidates and supporting the identification of suitable profiles.

8. **Interview Scheduling**

The system also provides functionality related to scheduling candidate interviews.

9. **DSA-Based Processing**

Several Data Structures and Algorithms are incorporated for text searching, similarity analysis, matching, and processing of resume information.

The project primarily focuses on supporting the recruitment screening process. It does not attempt to replace the complete recruitment process or make final hiring decisions. The system is intended to assist recruiters by organizing information and improving the efficiency of candidate screening.

1.6 Objectives

The main objective of TalentFlow is to develop an efficient Resume Search and Candidate Screening System that uses Data Structures and Algorithms to improve the processing and evaluation of candidate resumes.

The specific objectives of the project are:

- To develop a system for uploading and processing candidate resumes.
- To extract important information from resumes in a structured manner.
- To provide an organized method for managing candidate information.
- To implement efficient keyword-based resume searching.
- To apply string matching algorithms to resume text processing.
- To compare candidate profiles using resume similarity techniques.
- To support candidate matching based on relevant requirements.
- To provide candidate optimization and ranking functionality.
- To reduce the manual effort required for initial resume screening.
- To improve the speed of retrieving relevant candidate information.
- To provide an integrated frontend and backend application for recruiters.
- To demonstrate the practical application of Data Structures and Algorithms in a real-world recruitment system.

1.7 Proposed System

The proposed system, TalentFlow, is a full-stack Resume Search and Candidate Screening System designed to provide an organized approach to candidate management and resume analysis.

The system allows recruiters to upload candidate resumes through the application interface. The uploaded resumes are processed by the backend, where relevant information is extracted and made available for further operations. The system can work with PDF resumes and uses Apache PDFBox for extracting information from PDF documents.

After processing, candidate information can be searched and analyzed using the functionality provided by the system. Recruiters can search for candidates based on relevant keywords and requirements. The system also incorporates resume similarity and candidate matching techniques to assist in identifying suitable profiles.

The proposed system integrates multiple Data Structures and Algorithms into different operations. Algorithms such as Aho-Corasick, KMP, Rabin-Karp, Z-Function, Edit Distance, and Suffix Array support text-processing and searching requirements. Resume Similarity and Bipartite Matching support candidate comparison and matching, while Candidate Optimization and Scalable Resume Processing support broader candidate-processing requirements.

The application follows a client-server architecture in which the React frontend communicates with the Java Spring Boot backend through REST APIs. The frontend provides the user interface, while the backend manages resume processing, candidate information, search operations, matching services, and other system functions.

Proposed System Flow

Recruiter → Resume Upload → Resume Processing → Information Extraction → Candidate Data Storage → Search / Matching → Similarity Analysis → Candidate Ranking → Screening → Interview Scheduling

This flow represents the main functional path of the proposed system.

1.8 Advantages of the Proposed System

The proposed TalentFlow system provides several advantages over a completely manual resume screening process.

1.8.1 Reduced Screening Time

The system provides search and filtering mechanisms that help recruiters locate relevant candidates more quickly instead of manually examining every resume.

1.8.2 Efficient Resume Search

The integration of string-searching algorithms allows the system to perform efficient searches for relevant terms within resume information.

1.8.3 Organized Candidate Information

Candidate details extracted from resumes can be managed in a structured manner, making information easier to retrieve and compare.

1.8.4 Automated Resume Processing

The system processes uploaded resumes and extracts important information, reducing the amount of manual effort required to identify candidate details.

1.8.5 Candidate Comparison

Resume similarity and matching techniques provide mechanisms for comparing candidate profiles and identifying relevant candidates.

1.8.6 Application of Data Structures and Algorithms

The system demonstrates how different DSA techniques can be applied to practical problems involving text searching, similarity analysis, candidate matching, and optimization.

1.8.7 Integrated Web Application

The React frontend and Spring Boot backend provide an integrated environment through which recruiters can perform different resume and candidate management activities.

1.8.8 Improved Candidate Screening

By combining search, matching, similarity, and ranking capabilities, the system can assist recruiters in narrowing down candidates for further evaluation.

1.8.9 Centralized Recruitment Support

The system brings resume processing, candidate search, analysis, and related recruitment functions into a single application.

1.9 Applications

TalentFlow can be applied in different recruitment and candidate management environments where organizations need to process and screen multiple resumes.

1.9.1 Corporate Recruitment

Organizations can use the system to assist recruiters in searching and screening candidates for different job positions.

1.9.2 Human Resource Departments

Human Resource departments can use the system to organize candidate information and support the initial stages of recruitment.

1.9.3 Recruitment Agencies

Recruitment agencies that handle resumes for multiple job requirements can use resume search and candidate matching functionality to identify relevant profiles.

1.9.4 Campus Recruitment

The system can assist in managing and screening student resumes during campus recruitment activities.

1.9.5 Technical Recruitment

Organizations hiring candidates for technical positions can search resumes for relevant programming languages, technologies, skills, certifications, and experience.

1.9.6 Candidate Database Management

The system can be used as a structured platform for maintaining and retrieving candidate information from processed resumes.

1.9.7 Recruitment Screening Support

The system can assist recruiters during the initial screening stage by helping them identify candidates whose profiles are relevant to particular requirements.

1.10 Organization of the Report

This project report is organized into different chapters to provide a systematic explanation of the development, implementation, and evaluation of the TalentFlow Resume Search and Candidate Screening System.

Chapter 1 – Introduction presents the overview, background, problem statement, motivation, scope, objectives, proposed system, advantages, applications, and organization of the report.

Chapter 2 – System Requirements and Existing System discusses existing approaches to resume screening, resume search techniques, string matching, resume similarity, candidate matching and ranking, existing system limitations, and the research gap addressed by the proposed system.

Chapter 3 – System Design describes the functional and non-functional requirements, hardware and software requirements, technology stack, frontend and backend technologies, data management, development tools, system architecture, workflow, use case diagram, and data flow diagram.

Chapter 4 – DSA Algorithms and Implementation explains the Data Structures and Algorithms used in TalentFlow, including Aho-Corasick, KMP, Rabin-Karp, Z-Function, Edit Distance, Suffix Array, Resume Similarity, Bipartite Matching, Candidate Optimization, and Scalable Resume Processing. It also discusses their integration into the system and their computational

complexity.

Chapter 5 – Implementation describes the practical implementation of the system, including resume uploading, resume parsing, candidate management, candidate search, similarity analysis, duplicate detection, candidate optimization, interview scheduling, REST APIs, React frontend, and frontend-backend integration.

Chapter 6 – Results, Testing and Analysis presents the implemented system screens, functional results, algorithm testing, system testing, and performance analysis.

Chapter 7 – Challenges and Limitations discusses the difficulties encountered during development, the solutions applied to address those difficulties, and the limitations of the current system.

Chapter 8 – Conclusion and Future Work summarizes the project outcomes, discusses possible future improvements, and presents the technical skills gained during the development of TalentFlow.

Finally, the **References** section contains the books, documentation, tutorials, APIs, and other sources used during the development and documentation of the project.

2. SYSTEM REQUIREMENTS AND EXISTING SYSTEM

2.1 Introduction

TalentFlow is designed as a Resume Search and Candidate Screening System that supports the processing, management, searching, and screening of candidate resumes. The system is intended to reduce the effort associated with manually reviewing a large number of resumes by providing structured resume processing, candidate search, matching, similarity analysis, and screening capabilities.

The project combines a React-based frontend with a Java Spring Boot backend. The backend is responsible for resume processing, candidate information, search operations, matching services, and related functionality, while the frontend provides the interface through which recruiters interact with the system.

2.2 Existing Resume Screening Systems

In a conventional resume screening process, recruiters manually examine candidate resumes and identify information such as skills, education, work experience, certifications, and contact details. Candidates are then compared against the requirements of a job position. When the number of resumes is small, this process can be manageable; however, reviewing many resumes increases the time and effort required from recruiters.

Existing screening approaches can also involve basic keyword searching and filtering. These approaches help locate resumes containing particular terms, but they may provide limited support for deeper comparison between candidate profiles. TalentFlow extends the screening process by combining resume parsing, search, similarity, matching, ranking, and candidate-management functions in one application.

2.3 Resume Search Techniques

Resume search is an important function of TalentFlow because recruiters need to locate candidates according to skills and other relevant information. Keyword-based searching can be used to identify terms appearing in processed resume information. Efficient text-search techniques are particularly useful when the system needs to search through multiple candidate profiles.

TalentFlow incorporates several string-processing algorithms for this purpose. The project includes Aho-Corasick, Knuth-Morris-Pratt (KMP), Rabin-Karp, Z-Function, and Suffix Array implementations. These algorithms provide different approaches for locating patterns or organizing searchable text and form an important part of the project's DSA implementation.

2.4 String Matching Algorithms

String matching is the process of locating a pattern or keyword within a larger text. In a resume screening system, string matching can be applied to identify skills, technologies, qualifications, certifications, or other terms relevant to a job requirement.

TalentFlow includes multiple string matching techniques so that text-processing operations can be implemented using appropriate algorithmic approaches. Aho-Corasick is useful for searching

multiple patterns, while KMP uses a preprocessing-based approach for efficient single-pattern matching. Rabin-Karp uses hashing to compare patterns with text windows, and the Z-Function provides another linear-time pattern-processing technique. The Suffix Array provides an indexed representation of suffixes that can support text-search operations.

2.5 Resume Similarity Techniques

Resume similarity is used to determine how closely two pieces of resume information or candidate profiles correspond to each other. Similarity analysis can help identify candidates with related skills, experience, or other profile information.

TalentFlow includes a Resume Similarity component as part of its algorithmic functionality. The similarity operation supports candidate analysis and can be used together with other screening functions to help identify profiles that are relevant to a particular requirement.

Edit Distance is also included in the project. It provides a method for measuring the number of edit operations needed to transform one string into another and can therefore support comparison of textual information.

2.6 Candidate Matching and Ranking

Candidate matching involves comparing candidate information with the requirements used for screening. A recruiter may need to identify candidates who satisfy relevant skills or profile requirements and then compare the resulting candidates.

TalentFlow includes Bipartite Matching and Candidate Optimization components. Bipartite matching provides an algorithmic approach for establishing relationships between two sets of entities, while candidate optimization supports the processing and evaluation of candidate profiles. Together with search and similarity operations, these components contribute to the candidate screening workflow.

2.7 Existing System Limitations

A manual or basic resume-screening approach has limitations when the number of candidate resumes increases. The recruiter may need to repeatedly inspect documents, search for relevant terms, compare profiles, and organize candidate information.

- Manual review of a large number of resumes can require considerable time and effort.
- Candidate information may be difficult to retrieve when resumes are not organized in a structured form.
- Repeated keyword searches across individual documents can be inefficient.
- Manual comparison can make it difficult to apply a consistent screening process to many candidates.
- Basic keyword searching alone does not provide the complete similarity and matching functionality available in TalentFlow.

2.8 Research Gap

The project addresses the need for an integrated system that combines resume processing with Data Structures and Algorithms for searching, similarity analysis, candidate matching, and candidate optimization. Rather than treating these activities as separate manual operations, TalentFlow brings them together in a single full-stack application.

The project also demonstrates the practical use of several algorithmic techniques within a recruitment-related application. The abstract identifies Aho-Corasick, KMP, Rabin-Karp, Edit Distance, Z-Function, Suffix Array, Resume Similarity, Bipartite Matching, Candidate Optimization, and Scalable Resume Processing as components integrated into the system.

2.9 Summary

This chapter described the requirements and background of the TalentFlow system. It discussed resume screening, resume search, string matching, resume similarity, candidate matching, and ranking. It also identified the limitations of manual screening approaches and explained how TalentFlow combines multiple algorithmic techniques with a full-stack application to support an organized recruitment screening workflow.

3. SYSTEM DESIGN

3.1 Introduction

The system design of TalentFlow defines the structure, components, requirements, technologies, and workflow used to develop the Resume Search and Candidate Screening System. The design separates the user interface, application logic, resume processing, algorithmic operations, and data management so that the system can support the complete candidate screening workflow.

TalentFlow follows a full-stack architecture consisting of a React-based frontend and a Java Spring Boot backend connected through REST APIs. The frontend is responsible for user interaction, while the backend handles resume processing, candidate information, search operations, matching services, and related system functionality.

3.2 Functional Requirements

Functional requirements describe the operations that TalentFlow must provide to support recruiters during resume processing and candidate screening. The main functional requirements identified for the system are:

- The system shall allow recruiters to upload candidate resumes.
- The system shall process uploaded PDF resumes and extract relevant textual information.
- The system shall maintain candidate information obtained from processed resumes.
- The system shall provide candidate search functionality.
- The system shall support keyword-based searching of candidate information.
- The system shall provide resume similarity functionality for candidate comparison.
- The system shall support candidate matching and optimization operations.
- The system shall provide duplicate-related resume analysis functionality.
- The system shall provide interview scheduling functionality.
- The system shall expose backend functionality through REST APIs for communication with the frontend.
- The system shall provide a web-based interface through which recruiters can access the available modules.

3.3 Non-Functional Requirements

Non-functional requirements describe the quality and operational characteristics expected from the system. For TalentFlow, the important non-functional requirements include usability, maintainability, performance, reliability, scalability, and modularity.

- Usability: The application should provide a clear interface through which recruiters can perform

resume and candidate operations.

- **Performance:** Search and text-processing operations should use appropriate algorithms to avoid unnecessary processing.
- **Maintainability:** The frontend, backend, and algorithmic components should remain organized into manageable modules.
- **Scalability:** Resume-processing functionality should be structured so that the system can handle an increasing number of candidate resumes.
- **Reliability:** The system should process supported resume inputs and provide consistent results for supported operations.
- **Modularity:** Individual algorithms and application services should be separated so that they can be developed and tested independently.
- **Interoperability:** The frontend and backend should communicate through defined REST API interfaces.

3.4 Hardware Requirements

The project is a software-based web application and does not require specialized hardware for development or normal execution. A computer capable of running the Java backend, frontend development environment, database or data-management components, and web browser is sufficient.

- **Processor:** Modern multi-core processor.
- **Memory:** Sufficient RAM to run the development environment, Java application, frontend tooling, and browser simultaneously.
- **Storage:** Adequate storage for the source code, dependencies, resumes, and project data.
- **Display:** Standard monitor suitable for development and web application testing.
- **Network:** Network connectivity when required for development dependencies or application services.

3.5 Software Requirements

TalentFlow uses a combination of frontend, backend, document-processing, and development technologies. The software environment identified from the project documentation includes:

- **Operating System:** A desktop operating system capable of running the Java and frontend development environments.
- **Programming Language:** Java for the backend and algorithmic implementation.
- **Backend Framework:** Spring Boot.
- **Frontend Framework:** React.

- Document Processing: Apache PDFBox for extracting information from PDF resumes.
- API Communication: REST APIs between the frontend and backend.
- Development Environment: Visual Studio Code and the Java/frontend development toolchain used for the project.
- Web Browser: A modern browser for accessing and testing the frontend.

3.6 Technology Stack

TalentFlow uses a full-stack technology architecture. The frontend is implemented using React, while the backend is implemented using Java and Spring Boot. REST APIs provide communication between these two layers. Apache PDFBox is used for extracting information from PDF resumes. The algorithmic layer contains the DSA implementations used for searching, similarity analysis, matching, optimization, and scalable resume processing.

3.7 Frontend Technologies

The frontend of TalentFlow is developed using React. It provides the user interface through which recruiters can interact with the different functions of the system. The frontend is responsible for presenting candidate information, accepting resume uploads, providing search interfaces, displaying results, and providing access to other recruitment-related modules.

The React frontend communicates with the Spring Boot backend through REST APIs. This separation allows the presentation layer and application-processing layer to operate as distinct components.

3.8 Backend Technologies

The backend of TalentFlow is implemented using Java and Spring Boot. It provides the application logic required for resume processing, candidate management, search, matching, similarity analysis, and related services.

Apache PDFBox is used for processing PDF resumes and extracting text from supported PDF documents. The backend also contains separate implementations of the project's Data Structures and Algorithms, allowing algorithmic operations to be integrated with the application services.

REST APIs are used to expose backend functionality to the React frontend. This provides a clear communication interface between the user interface and server-side processing.

3.9 Database / Data Management

TalentFlow requires organized management of candidate and resume information so that processed candidate data can be retrieved during search, matching, comparison, and screening operations. The system separates candidate information from the user-interface layer and processes it through backend services.

The data-management layer supports the storage and retrieval of candidate-related information generated from processed resumes. This allows recruiters to access candidate profiles without

repeatedly processing the same resume for every search operation.

3.10 Development Tools

The development of TalentFlow requires tools for writing, building, testing, and running both the backend and frontend components. The project uses a Java/Spring Boot development environment for the backend and a React development environment for the frontend. Visual Studio Code can be used as the development editor for managing the project source code and running development commands.

Additional libraries and dependencies are managed as part of the respective frontend and backend projects. The backend project uses the standard Java build and dependency-management configuration defined in its project files.

3.11 System Architecture

TalentFlow follows a layered full-stack architecture. The major architectural components are the React frontend, REST API layer, Spring Boot backend services, resume-processing and DSA algorithm layer, and candidate data-management layer.

The recruiter interacts with the React frontend. User actions such as resume upload, candidate search, similarity analysis, and scheduling requests are sent to the backend through REST APIs. The Spring Boot backend receives the requests and invokes the appropriate services. Resume-processing services extract information from supported PDF documents, while the algorithmic components perform searching, similarity, matching, optimization, or other required operations. The resulting information is then returned to the frontend for presentation.

The architecture therefore provides a separation of concerns between presentation, communication, application services, algorithmic processing, and data management.

3.12 System Workflow

The overall workflow begins when a recruiter interacts with the TalentFlow frontend. A resume can be uploaded through the user interface and sent to the backend for processing. The backend extracts the available text from the supported resume document and processes the resulting information.

After processing, candidate information can be stored and accessed through the application. Recruiters can then search candidate profiles, apply relevant screening criteria, compare profiles using similarity operations, and use candidate matching or optimization functionality. The system also provides interview scheduling functionality as part of the recruitment workflow.

The main workflow can be represented as:

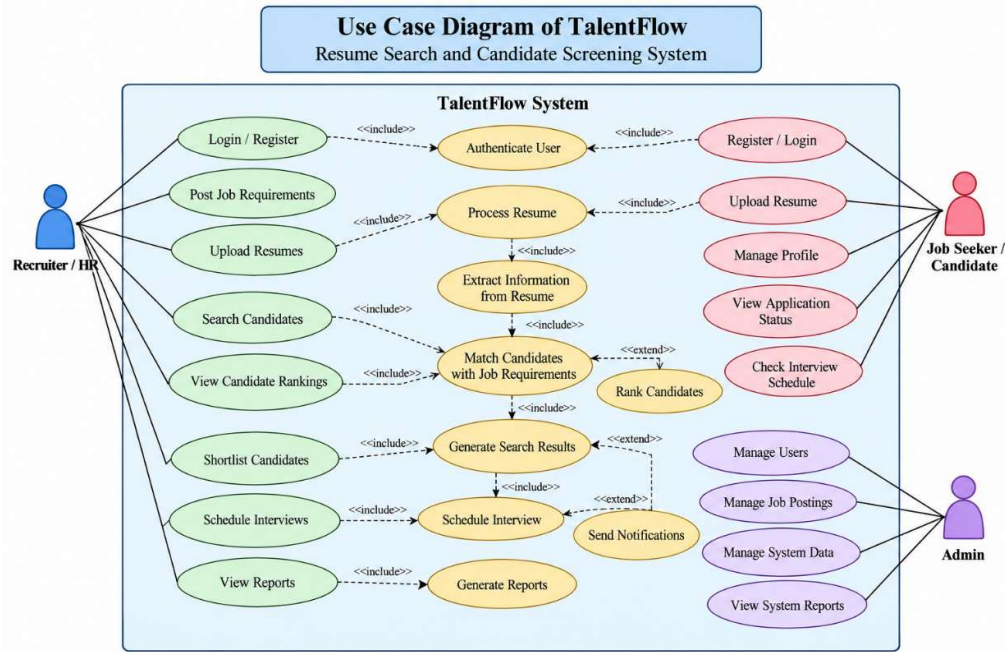
Recruiter → React Frontend → REST API → Spring Boot Backend → Resume Processing / DSA Algorithms → Candidate Data → Search / Matching / Screening → Results

3.13 Use Case Diagram

The use case model of TalentFlow represents the main interactions between the recruiter and the

system. The recruiter is the primary user who accesses the application to perform candidate and resume-related tasks.

The major use cases include uploading a resume, processing resume information, viewing candidate details, searching candidates, performing similarity analysis, checking duplicate-related information, using candidate optimization and matching functions, and scheduling interviews.

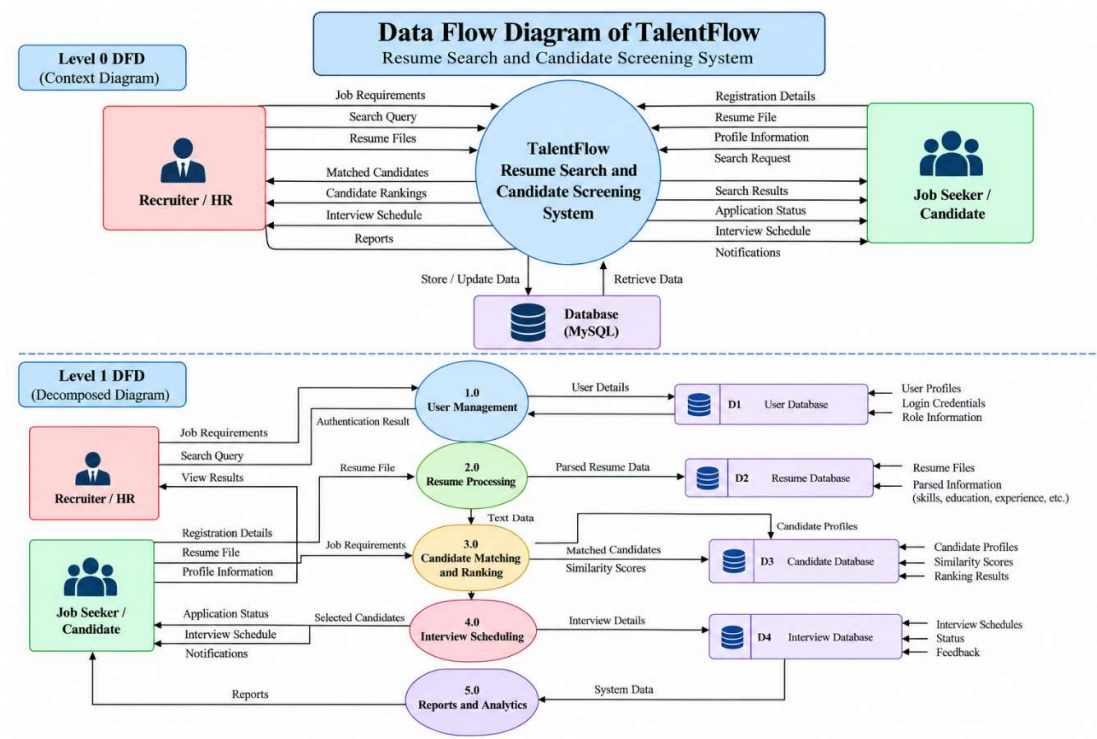


The use case diagram should be placed here as Figure 3.1 and should show the Recruiter as the main actor connected to the major TalentFlow functions.

3.14 Data Flow Diagram

The Data Flow Diagram represents how information moves through the TalentFlow system. Resume information enters the system through the upload interface and is transferred to the backend for processing. The resume-processing component extracts relevant information, after which candidate data can be stored and accessed by search, similarity, matching, optimization, and scheduling services.

The results generated by the backend are returned through the REST API to the React frontend, where they are presented to the recruiter. The DFD should therefore show the flow of resume data from the recruiter to the frontend, backend processing components, candidate data management, algorithmic services, and finally the results returned to the recruiter.



The Data Flow Diagram should be placed here as Figure 3.2.

3.15 Summary

This chapter described the system design of TalentFlow, including its functional and non-functional requirements, hardware and software requirements, technology stack, frontend and backend technologies, data management, and development environment. The chapter also described the system architecture, workflow, use case model, and data flow. The design provides the foundation for implementing the DSA algorithms and application modules described in the following chapters.

4. DSA ALGORITHMS AND IMPLEMENTATION

4.1 Introduction

A major objective of TalentFlow is to demonstrate the practical application of Data Structures and Algorithms to a real-world recruitment problem. The system incorporates multiple algorithms for text searching, text comparison, resume similarity, candidate matching, candidate optimization, and scalable resume processing.

The project includes implementations of Aho-Corasick, Knuth-Morris-Pratt (KMP), Rabin-Karp, Z-Function, Edit Distance, Suffix Array, Resume Similarity, Bipartite Matching, Candidate Optimization, and Scalable Resume Processing. These algorithms are integrated into the backend to support different stages of candidate search and screening.

4.2 Aho-Corasick Algorithm

The Aho-Corasick algorithm is a multiple-pattern string matching technique. It is useful when several keywords need to be searched within the same text. In a resume screening application, this type of operation can be applied when multiple skills, technologies, or other recruitment-related keywords need to be identified.

The algorithm constructs a trie containing the search patterns and uses failure links to continue matching when a direct transition is not available. This allows multiple patterns to be processed efficiently in a single search operation.

In TalentFlow, the Aho-Corasick implementation forms part of the DSA-based text-search functionality used for resume processing and candidate search.

4.3 Knuth-Morris-Pratt (KMP) Algorithm

The Knuth-Morris-Pratt algorithm is a string matching algorithm that avoids unnecessary rechecking of characters after a mismatch. It preprocesses the pattern and creates information that can be used to determine the next position from which matching should continue.

For TalentFlow, KMP can be applied to search for a particular skill, qualification, technology, or other keyword within processed resume text. Its inclusion demonstrates the use of pattern preprocessing to improve string searching.

4.4 Rabin-Karp Algorithm

The Rabin-Karp algorithm uses hashing to compare a pattern with portions of a larger text. Instead of comparing every character of the pattern at every position, the algorithm first compares hash values and performs a direct comparison when the hash values indicate a possible match.

Within TalentFlow, Rabin-Karp provides another algorithmic approach for searching textual information extracted from resumes. It demonstrates the use of hashing for pattern matching and provides an alternative to direct character-by-character searching.

4.5 Z-Function Algorithm

The Z-Function is a string-processing technique that calculates, for each position in a string, the length of the longest substring starting at that position that is also a prefix of the complete string.

The technique can be used to solve pattern matching problems by combining the pattern and text into a single string and examining the resulting Z-values. In TalentFlow, the Z-Function implementation contributes to the project's collection of text-searching algorithms.

4.6 Edit Distance Algorithm

Edit Distance measures the minimum number of basic edit operations required to transform one string into another. The operations commonly considered by the algorithm are insertion, deletion, and substitution.

In a resume screening context, Edit Distance can support comparison of textual information when two terms are similar but not exactly identical. This makes it useful as part of text comparison and similarity-related processing.

TalentFlow includes an Edit Distance implementation as one of its DSA components, allowing the project to demonstrate dynamic-programming-based string comparison in a practical application.

4.7 Suffix Array

A Suffix Array is an indexed arrangement of the suffixes of a string. By organizing suffixes in sorted order, the structure can support efficient searching and other operations on large text.

In TalentFlow, the Suffix Array implementation contributes to the system's text-processing and searching capabilities. Its inclusion demonstrates the use of an advanced text-indexing data structure in a recruitment application.

4.8 Resume Similarity

Resume Similarity is used to compare resume information and determine how closely candidate profiles correspond to one another or to relevant screening information. Similarity analysis can help recruiters identify candidates with related skills, experience, or other profile characteristics.

The Resume Similarity component is integrated with the other DSA-based processing functions in TalentFlow. It supports candidate analysis and can be used as part of the screening workflow to assist in comparing profiles.

The project abstract identifies Resume Similarity as one of the algorithmic techniques incorporated into the backend for efficient similarity analysis and candidate processing.

4.9 Bipartite Matching

Bipartite Matching is a graph-based technique for establishing matches between elements belonging to two separate sets. A bipartite graph contains two disjoint sets of vertices, with edges representing possible relationships between the sets.

In TalentFlow, Bipartite Matching is included as a candidate-matching algorithm. It can be used

to represent relationships between candidate-related entities and job or requirement-related entities, supporting the project's objective of applying graph algorithms to recruitment-related matching.

4.10 Candidate Optimization and Ranking

Candidate Optimization is used to support the evaluation and organization of candidate profiles according to relevant screening information. Candidate ranking can help recruiters focus on profiles that are more relevant to the requirements being considered.

The Candidate Optimization component is part of the DSA functionality of TalentFlow. It works together with search, similarity, and matching operations to support the candidate screening process.

The system is intended to assist recruiters in evaluating candidates; the ranking or optimization process does not replace the recruiter's final decision.

4.11 Scalable Resume Processing

Scalable Resume Processing addresses the need to process resume information in a manner that can support an increasing number of candidate documents. Efficient processing becomes important when a recruitment system needs to handle many resumes rather than a single document.

TalentFlow includes a Scalable Resume Processing component as part of its backend DSA implementation. It is intended to support the efficient processing of resume information and complement the search, similarity, matching, and candidate-management functions.

4.12 Algorithm Integration in TalentFlow

The DSA algorithms are not implemented as isolated academic examples. They are incorporated into the backend as components supporting different recruitment-related operations.

- Aho-Corasick: Multiple keyword or pattern searching.
- KMP: Single-pattern string searching.
- Rabin-Karp: Hash-based pattern matching.
- Z-Function: Pattern-processing and string matching.
- Edit Distance: Text comparison and similarity-related processing.
- Suffix Array: Text indexing and searching.
- Resume Similarity: Comparison of resume information.
- Bipartite Matching: Candidate-related matching operations.
- Candidate Optimization: Candidate evaluation and optimization.
- Scalable Resume Processing: Processing resume information at a larger scale.

The combination of these components demonstrates how different algorithmic approaches can be selected for different stages of a recruitment screening application.

4.13 Complexity Analysis

The computational complexity of an algorithm depends on its implementation and on the input size. Therefore, the exact performance of each implementation should be reported according to the source code and the test cases used in the project.

For the final report, complexity analysis should distinguish between preprocessing and query/search operations where applicable. It should also specify the relevant input variables, such as text length, pattern length, number of patterns, number of candidates, and graph size.

The following qualitative comparison can be used to organize the analysis:

- Aho-Corasick: Designed for efficient multiple-pattern matching after construction of the pattern automaton.
- KMP: Provides linear-time pattern matching after preprocessing the pattern.
- Rabin-Karp: Uses rolling hash values to compare pattern-sized text windows.
- Z-Function: Supports linear-time string processing for a string of length n .
- Edit Distance: The standard dynamic-programming formulation uses a table based on the lengths of the two input strings.
- Suffix Array: Requires construction of an index over the suffixes and supports efficient indexed text operations.
- Bipartite Matching: Complexity depends on the matching algorithm and the number of vertices and edges in the graph.
- Candidate Optimization: Complexity depends on the optimization strategy and the number of candidates and attributes processed.
- Scalable Resume Processing: Performance depends on the number and size of resumes and the processing strategy used.

Exact numerical complexity values and benchmark results should be added to the final report only after they are confirmed from the implemented source code and measured test results.

4.14 Summary

This chapter presented the Data Structures and Algorithms implemented in TalentFlow. The system includes multiple string-matching techniques, text comparison, indexing, similarity analysis, graph-based matching, candidate optimization, and scalable resume processing. These components demonstrate the practical use of DSA concepts within a complete recruitment-oriented application.

5. IMPLEMENTATION

5.1 Introduction

This chapter describes the practical implementation of the TalentFlow Resume Search and Candidate Screening System. The implementation connects the system design and Data Structures and Algorithms described in the previous chapters with the application workflow used for resume processing and candidate screening. The major implementation areas are resume uploading, PDF resume parsing, candidate information management, candidate search, similarity analysis, duplicate-related analysis, candidate optimization, interview scheduling, REST API communication, the React frontend, and frontend-backend integration.

TalentFlow is described in the project documentation as a full-stack application in which a React frontend communicates with a Java Spring Boot backend through REST APIs. The backend handles resume processing and candidate-related services, while the frontend provides the interface through which the recruiter interacts with the system.

5.2 Implementation Architecture

The implementation follows the layered architecture defined in the system design. The recruiter interacts with the React frontend. User operations such as resume upload, candidate search, similarity analysis, and scheduling requests are communicated to the Spring Boot backend through REST APIs. The backend invokes the required application services, resume-processing functions, and DSA components. The processed information is then returned to the frontend for presentation.

The implementation can therefore be viewed as a sequence of cooperating layers: presentation, REST communication, backend services, resume-processing and algorithmic processing, and candidate data management. This separation allows the individual functions to be developed and integrated as distinct components.

5.3 Resume Upload Module

The Resume Upload module provides the entry point for candidate documents. The recruiter selects or uploads a resume through the application interface. The uploaded document is transferred to the backend for processing. The project documentation specifies PDF resumes as the supported resume format for the parsing workflow.

The upload stage is important because subsequent candidate operations depend on the information obtained from the uploaded document. A successful upload therefore initiates the resume-processing pipeline rather than requiring the recruiter to manually enter every candidate detail.

Recruiter selects a candidate resume.

The resume is submitted through the application interface.

The backend receives the uploaded document.

The document is passed to the resume-processing component.

Extracted candidate information becomes available for subsequent operations.

5.4 Resume Parsing and Information Extraction

TalentFlow uses Apache PDFBox for reading and extracting text from PDF resumes. The purpose of this module is to convert unstructured resume content into information that can be processed by the candidate-management and DSA components.

The documented processing sequence is: Resume PDF → PDFBox → Extracted Text → Candidate Object. The candidate information described in the project includes details such as name, skills, education, experience, projects, certifications, and contact-related information, depending on the information available in the resume.

After extraction, the resulting information can be passed to the algorithmic processing layer. This creates a clear boundary between document handling and the DSA operations used for searching, comparison, matching, and ranking.

5.5 Candidate Information Management

The Candidate Management module organizes information obtained from processed resumes. Instead of requiring the recruiter to repeatedly inspect the original PDF for every operation, candidate information can be represented in a structured form and accessed through the application.

The project documentation describes Candidate objects containing information such as name, skills, education, experience, and projects. This structured representation provides the input required for candidate search, similarity analysis, matching, and ranking operations.

Candidate identity and basic information

Skills and technical information

Educational qualifications

Work experience

Projects and other relevant profile information

5.6 Candidate Search Module

The Candidate Search module allows the recruiter to enter relevant keywords or search criteria and retrieve candidate profiles containing matching information. Search is supported by the string matching algorithms described in Chapter 4.

TalentFlow includes Aho-Corasick, Knuth-Morris-Pratt (KMP), Rabin-Karp, Z-Function, and Suffix Array components for text-searching and text-indexing purposes. The exact algorithm used for a particular operation depends on the corresponding implementation and integration in the project source code.

The search process can be represented as: Search Query → Candidate Information → Pattern

Matching → Matching Candidates → Candidate Ranking. The search result is then presented to the recruiter through the frontend.

5.7 Resume Similarity and Text Comparison

Resume Similarity provides a mechanism for comparing candidate information and identifying profiles with related content. It can be used together with search and candidate matching to support the screening workflow.

Edit Distance is included as a text-comparison technique. It measures the number of edit operations required to transform one string into another and can therefore support comparisons where textual variations or spelling differences occur. Similarity-related processing should be interpreted as support for recruiter evaluation rather than as an automatic replacement for the recruiter's final decision.

5.8 Duplicate Detection

Duplicate-related analysis is included as one of the functions identified in the system workflow. Its purpose is to help identify potentially repeated or closely related candidate information so that the recruiter can work with a more organized candidate set.

The project documentation identifies duplicate checking as a candidate-related use case, but it does not provide sufficient source evidence in the available project material to state one specific duplicate-detection formula or threshold. Therefore, the exact duplicate criterion should be described according to the implementation source code and test results when those details are documented.

5.9 Candidate Matching and Optimization

Candidate Matching compares candidate information with the requirements used for screening. The system includes Bipartite Matching and Candidate Optimization as algorithmic components. These components operate together with search and similarity functions to support candidate evaluation.

Candidate optimization and ranking are intended to help recruiters identify relevant profiles and review candidates in an organized order. The system is a decision-support application: the ranking or optimization result assists screening but does not replace the recruiter's final hiring decision.

5.10 Candidate Ranking

Candidate ranking presents relevant candidates in an order that supports recruiter review. The ranking stage can use information produced by search, similarity, matching, and optimization operations. A candidate with stronger correspondence to the screening requirements can therefore be presented for earlier review.

The available project documentation does not establish a single fixed numerical ranking formula that should be reported as a measured implementation result. Consequently, the final report should use the exact scoring equation and weights only if they are confirmed from the implemented source code.

5.11 Interview Scheduling Module

Interview Scheduling is included as part of the recruitment workflow. After candidates have been identified and reviewed, the recruiter can use the scheduling functionality to support the next stage of the recruitment process.

The scheduling function complements the screening workflow by connecting candidate evaluation with a subsequent recruitment activity. It is not part of the DSA search process itself, but it is included in the overall application workflow described for TalentFlow.

5.12 REST API Implementation

The frontend and backend communicate through REST APIs. The React frontend sends requests for operations such as resume processing, candidate search, candidate-related analysis, and scheduling. The Java Spring Boot backend receives these requests, performs the corresponding service operations, and returns the results to the frontend.

This API-based separation allows the presentation layer and backend processing layer to remain independent. It also provides a structured communication mechanism for transferring resume-related requests and candidate information between the application components.

5.13 React Frontend Implementation

The React frontend provides the recruiter-facing interface of TalentFlow. It is responsible for presenting application functions and displaying information returned by the backend. The documented frontend workflow includes resume upload, candidate search, candidate information access, and presentation of screening-related results.

The frontend communicates with the backend rather than performing the core resume-processing and algorithmic operations itself. This maintains the separation of responsibilities established by the system architecture.

5.14 Spring Boot Backend Implementation

The Java Spring Boot backend forms the application-processing layer of TalentFlow. It receives requests from the frontend and coordinates resume processing, candidate information handling, search operations, matching, similarity-related processing, and other recruitment functions.

The backend also provides the environment in which the DSA components are integrated with the application. This integration is important because the algorithms are not treated only as isolated academic implementations; they are intended to support actual recruitment-related operations in the system.

5.15 Frontend-Backend Integration

Frontend-backend integration connects the user interface with the processing services. A typical operation begins with an action performed by the recruiter in the React interface. The corresponding request is sent through a REST API to the Spring Boot backend. The backend performs the required processing and returns the result, which is then displayed in the frontend.

For resume screening, the integrated flow can be summarized as follows:

Recruiter interacts with the React frontend.

The frontend sends a request through the REST API.

The Spring Boot backend receives and processes the request.

Resume-processing or DSA services perform the required operation.

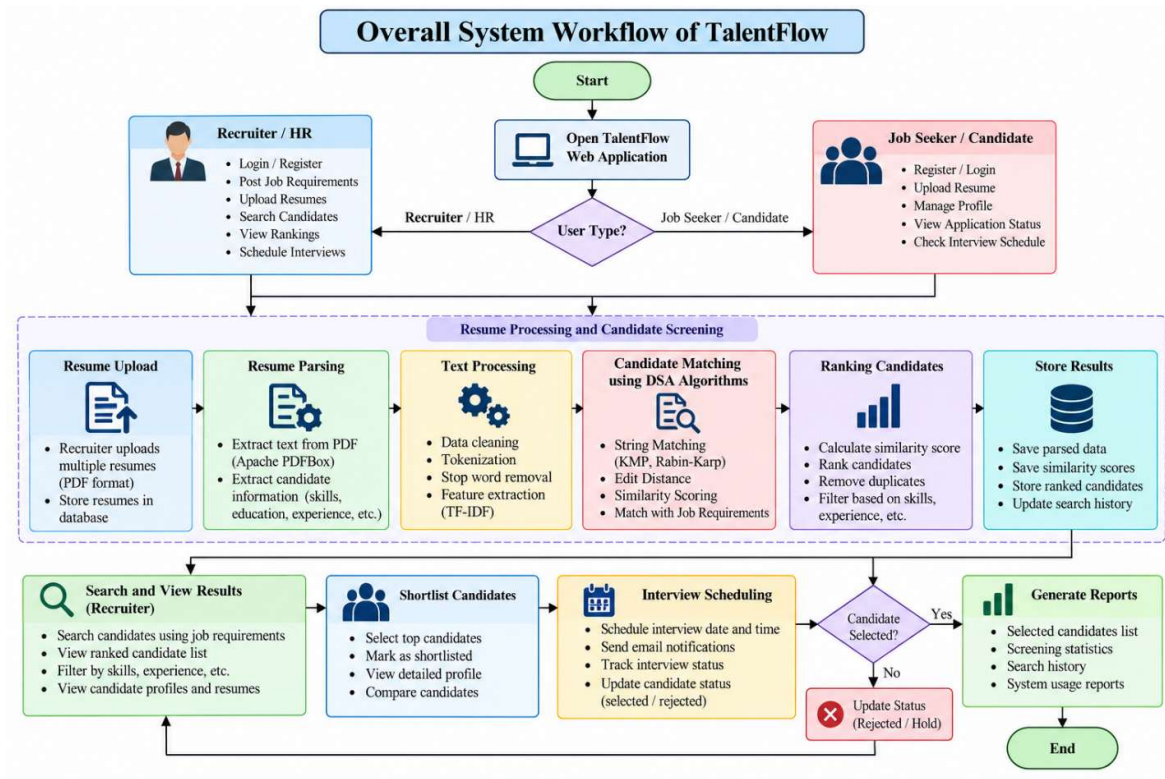
Candidate information or screening results are returned.

The React frontend displays the result to the recruiter.

5.16 Overall Implementation Flow

The complete implementation combines the modules described above into a single recruitment screening workflow. The documented system flow begins with recruiter interaction and resume upload, continues through PDF text extraction and structured candidate information, and then applies search, comparison, matching, ranking, and related candidate-management functions.

Step	Implementation Component	Purpose
1	React Frontend	Recruiter interaction and resume upload
2	REST API	Transfers requests between frontend and backend
3	Spring Boot Backend	Coordinates application services
4	Apache PDFBox	Extracts text from PDF resumes
5	Candidate Representation	Organizes extracted candidate information
6	DSA Services	Search, similarity, matching, and optimization
7	Results Processing	Produces candidate-related screening results



5.17 Implementation Summary

This chapter described the practical implementation of TalentFlow from resume upload to recruiter-facing results. The implementation uses a React frontend and Java Spring Boot backend connected through REST APIs. Apache PDFBox supports PDF text extraction, while the DSA layer provides string searching, text comparison, similarity, matching, optimization, and scalable processing components.

The major implementation modules—resume upload, parsing, candidate management, search, similarity, duplicate-related analysis, candidate optimization, ranking, and interview scheduling—work together to support the recruitment screening workflow. The next chapter presents the results, testing, and analysis of the implemented system.

6. RESULTS, TESTING AND ANALYSIS

6.1 Introduction

This chapter presents the results, testing approach, and analysis of the TalentFlow Resume Search and Candidate Screening System. The project documentation identifies this chapter as the place for implemented system screens, functional results, algorithm testing, system testing, and performance analysis.

The available project material defines the expected workflow from recruiter interaction and resume upload through PDF text extraction, candidate representation, search, similarity-related processing, matching, ranking, and presentation of results. Where measured execution results are not available in the supplied project material, this chapter records the expected or documented result rather than inventing benchmark values.

6.2 System Execution Flow

The implemented workflow begins with the recruiter interacting with the application. A resume can be uploaded and passed to the backend for processing. Apache PDFBox is used for PDF text extraction, after which candidate information is organized for subsequent search and screening operations.

The documented DSA workflow includes pattern searching, approximate text comparison, candidate matching, and ranking. The resulting candidate information is returned to the frontend and presented to the recruiter.

Stage	Input	Expected/Documented Result
1. Resume Upload	PDF resume	Resume received for processing
2. PDF Processing	Uploaded PDF	Text extracted using PDFBox
3. Candidate Representation	Extracted text	Structured candidate information
4. Search	Recruiter query	Relevant candidate information identified
5. Text Comparison	Search text / candidate text	Possible textual variations considered
6. Matching	Candidate information and requirements	Candidate relevance determined
7. Ranking	Matching information	Candidates arranged by relevance
8. Presentation	Processed result	Results displayed to recruiter

6.3 Functional Testing

Functional testing checks whether the major system functions behave according to their intended purpose. The project documentation identifies resume upload, PDF text extraction, candidate object creation, skill searching, spelling-tolerant searching, candidate matching, candidate ranking, ranked results, and candidate details as expected system outcomes.

Resume upload: verify that a supported PDF resume can be submitted for processing.

PDF text extraction: verify that text available in the PDF is extracted for further processing.

Candidate information: verify that extracted information is represented as candidate data.

Keyword search: verify that a relevant search term can be located in candidate information.

Spelling-tolerant search: verify that text comparison can assist when search terms contain variations.

Candidate matching: verify that candidate information can be compared with screening requirements.

Candidate ranking: verify that candidates can be ordered according to the implemented relevance logic.

Candidate details: verify that the recruiter can inspect the selected candidate information.

6.4 Resume Upload Testing

Resume upload is the first functional stage of the documented processing pipeline. A successful test should confirm that the application accepts the intended PDF input and passes it to the resume-processing component. The project material specifically describes PDF resume upload as a system requirement.

The exact number of files tested, file sizes, and measured upload times are not specified in the available project documentation. These values should therefore be added only from actual test execution.

6.5 PDF Text Extraction Testing

PDF text extraction is performed using Apache PDFBox. The expected result of this test is extracted text that can be passed to the candidate-information processing stage. The project documentation gives the processing sequence as Resume PDF → PDFBox → Extracted Text → Candidate Object.

A complete implementation report should record successful extraction for the actual sample resumes used by the project. No unsupported extraction accuracy percentage is stated here.

6.6 Candidate Search Testing

Candidate search is tested by entering relevant skills or keywords and checking whether the corresponding candidate information is identified. The project documentation describes KMP as

a pattern-searching technique that can be used to search for skills or keywords inside extracted resume text.

The project material also describes multiple string-processing algorithms, including Aho-Corasick, KMP, Rabin-Karp, Z-Function, and Suffix Array. Their exact runtime measurements should be reported only when obtained from the implemented programs and test environment.

6.7 Edit Distance Testing

Edit Distance is used to compare strings and can support searches affected by spelling variations. The project documentation gives examples such as comparing a misspelled technology name with its intended spelling. The standard dynamic-programming implementation described in the project has $O(m \times n)$ time complexity and $O(m \times n)$ space complexity.

Testing should verify that insertion, deletion, and substitution differences are handled according to the implemented Edit Distance logic.

6.8 Candidate Matching and Ranking Testing

Candidate matching combines candidate information with the recruiter's screening requirements. The project documentation states that skills, education, experience, and projects can contribute to the matching process. Candidates are then assigned relevance scores and displayed in ranked order.

An example in the project material uses candidate scores of 96%, 91%, and 84% to illustrate ranked output. These values are examples from the project documentation and should not be presented as measured final-system performance unless they were actually generated by the integrated implementation.

6.9 Algorithm Testing

Algorithm testing verifies the correctness of the individual DSA components before or during their integration with the application. The project identifies Aho-Corasick, KMP, Rabin-Karp, Z-Function, Edit Distance, Suffix Array, Resume Similarity, Bipartite Matching, Candidate Optimization, and Scalable Resume Processing as algorithmic components.

Pattern-search algorithms should be checked using known text-pattern pairs.

Edit Distance should be checked using strings with insertion, deletion, and substitution differences.

Similarity functions should be checked using candidate text with known levels of textual overlap.

Matching functions should be checked using candidate-requirement combinations with expected relationships.

Ranking should be checked to ensure that the output order follows the implemented scoring logic.

6.10 System Integration Testing

Integration testing verifies communication between the React frontend, REST API layer, Spring Boot backend, resume-processing services, algorithmic services, and candidate data-management layer.

Frontend request is sent correctly to the REST API.

Backend receives and processes the request.

PDF processing produces information for downstream services.

DSA services receive the required candidate or query information.

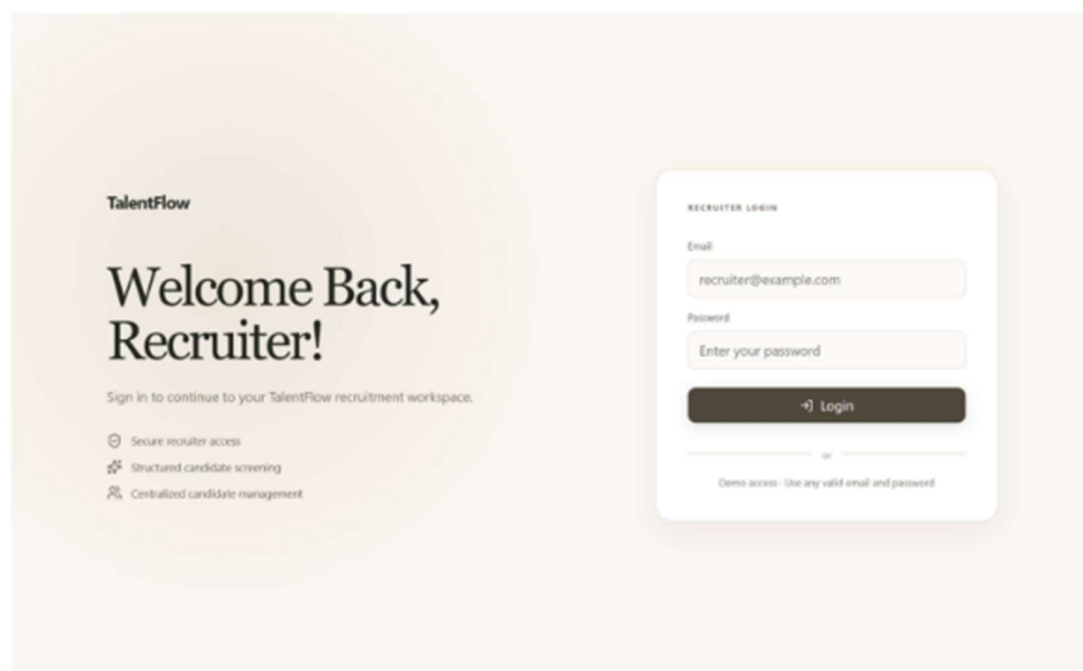
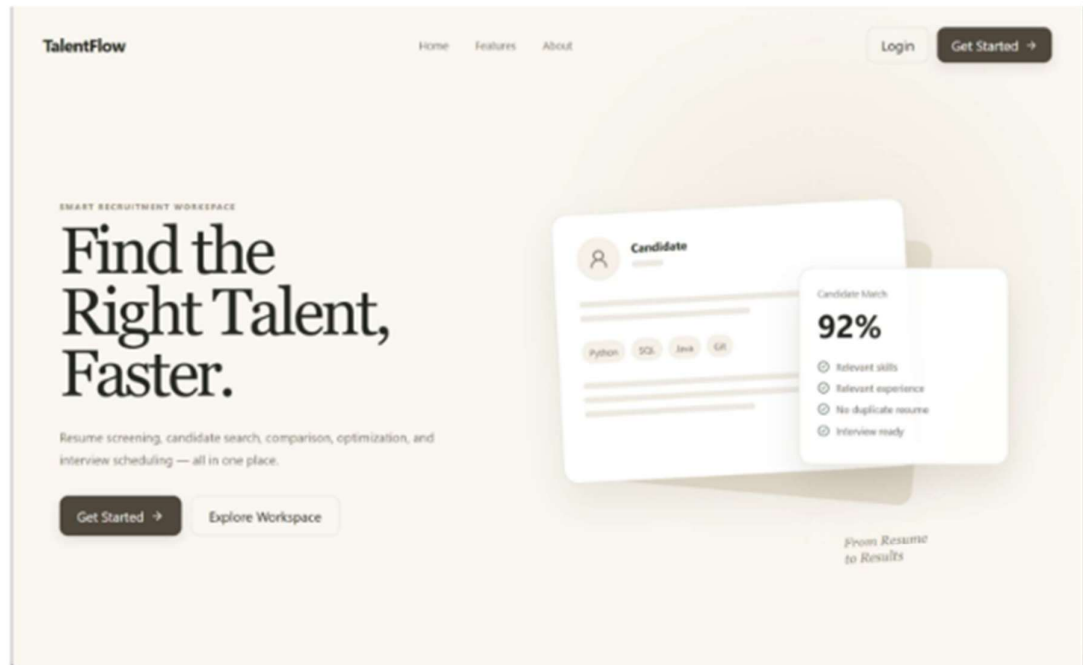
Backend returns the processed result.

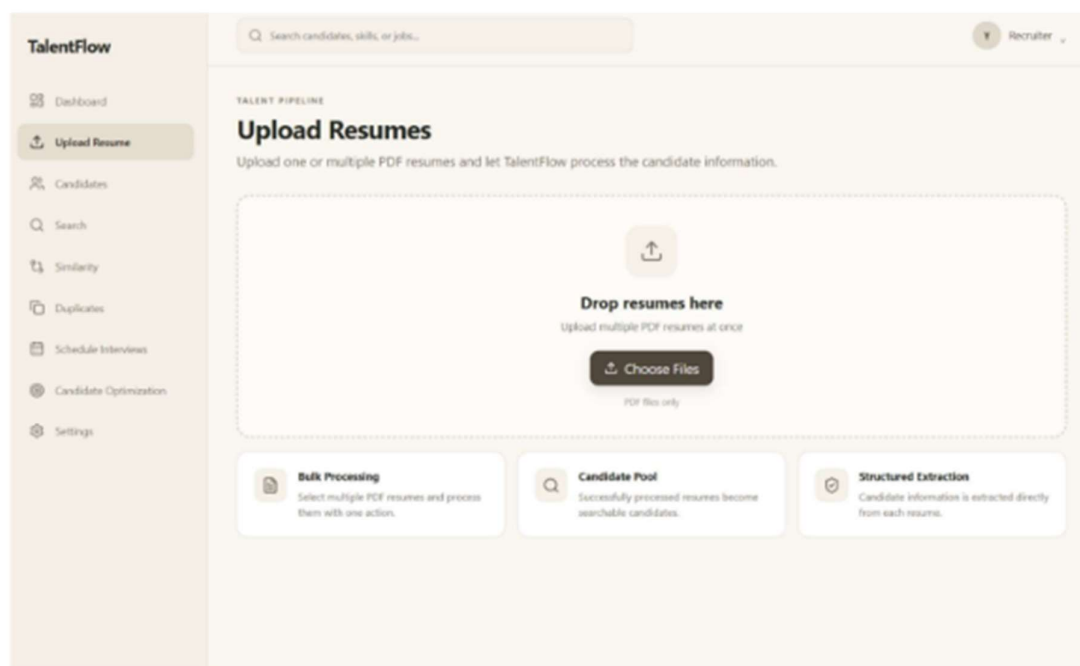
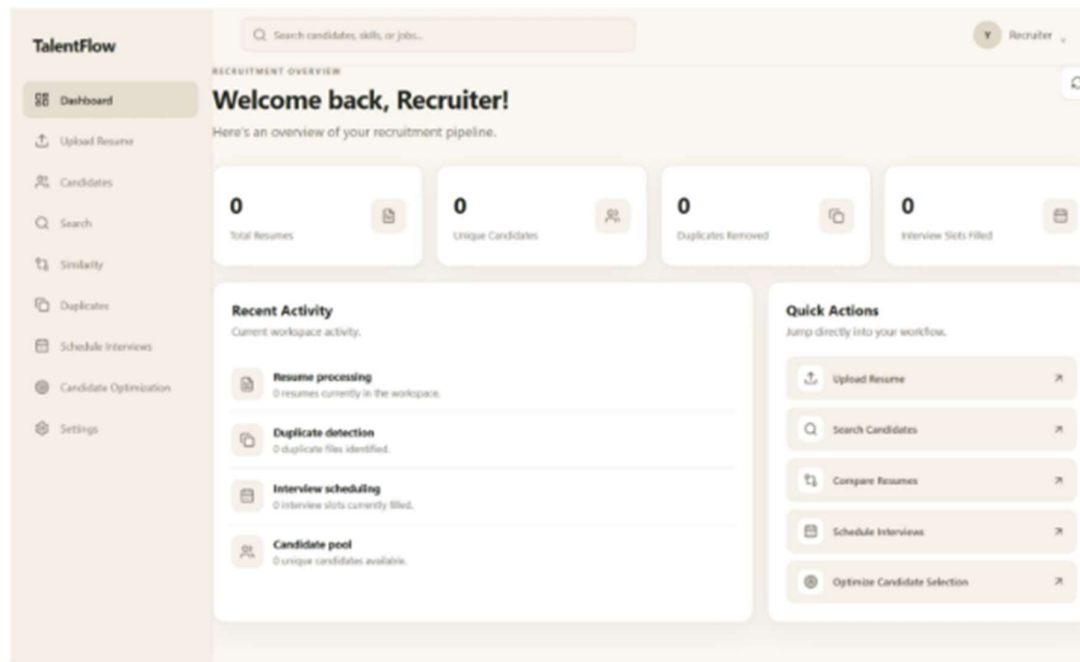
Frontend displays the returned result.

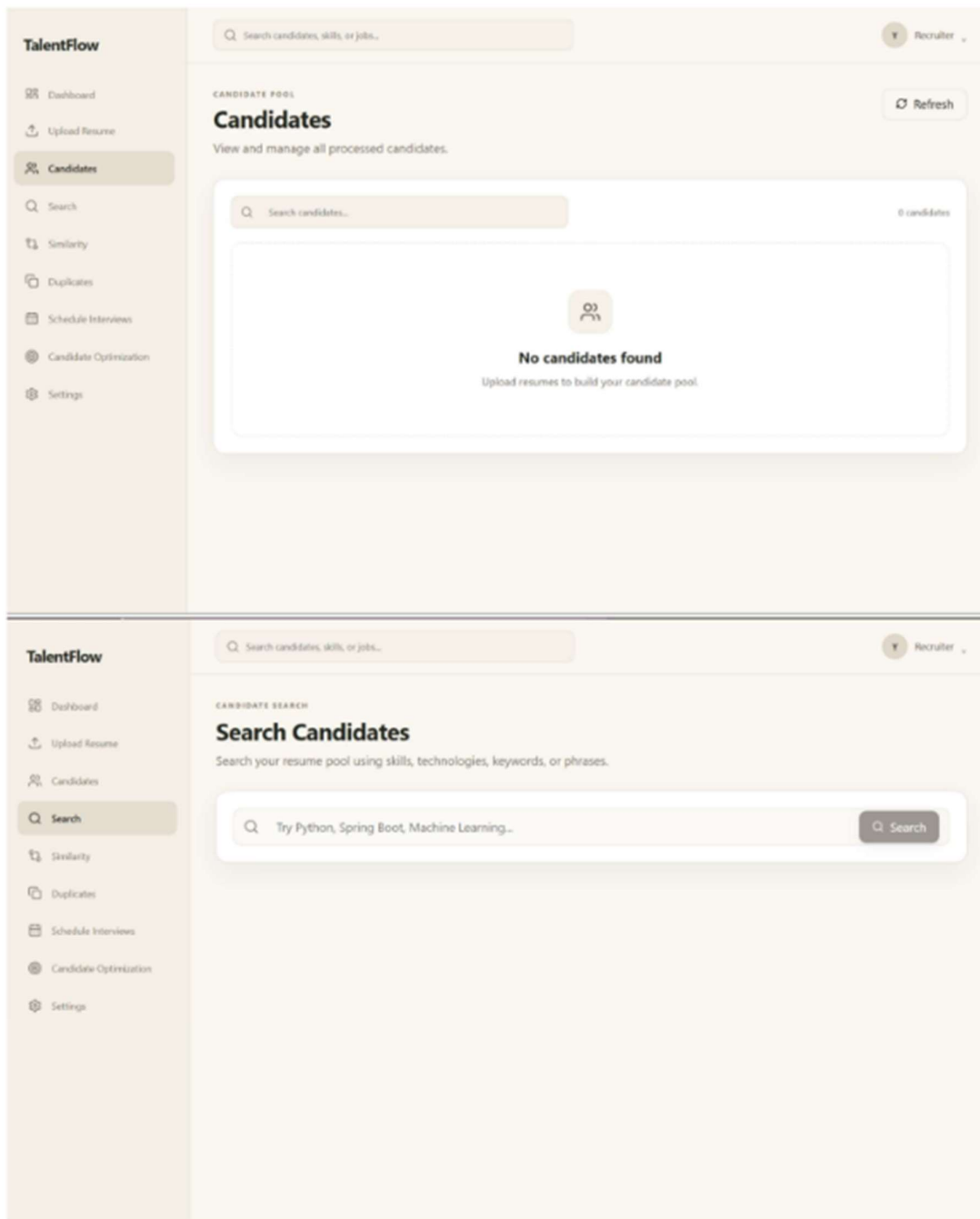
6.11 User Interface Results

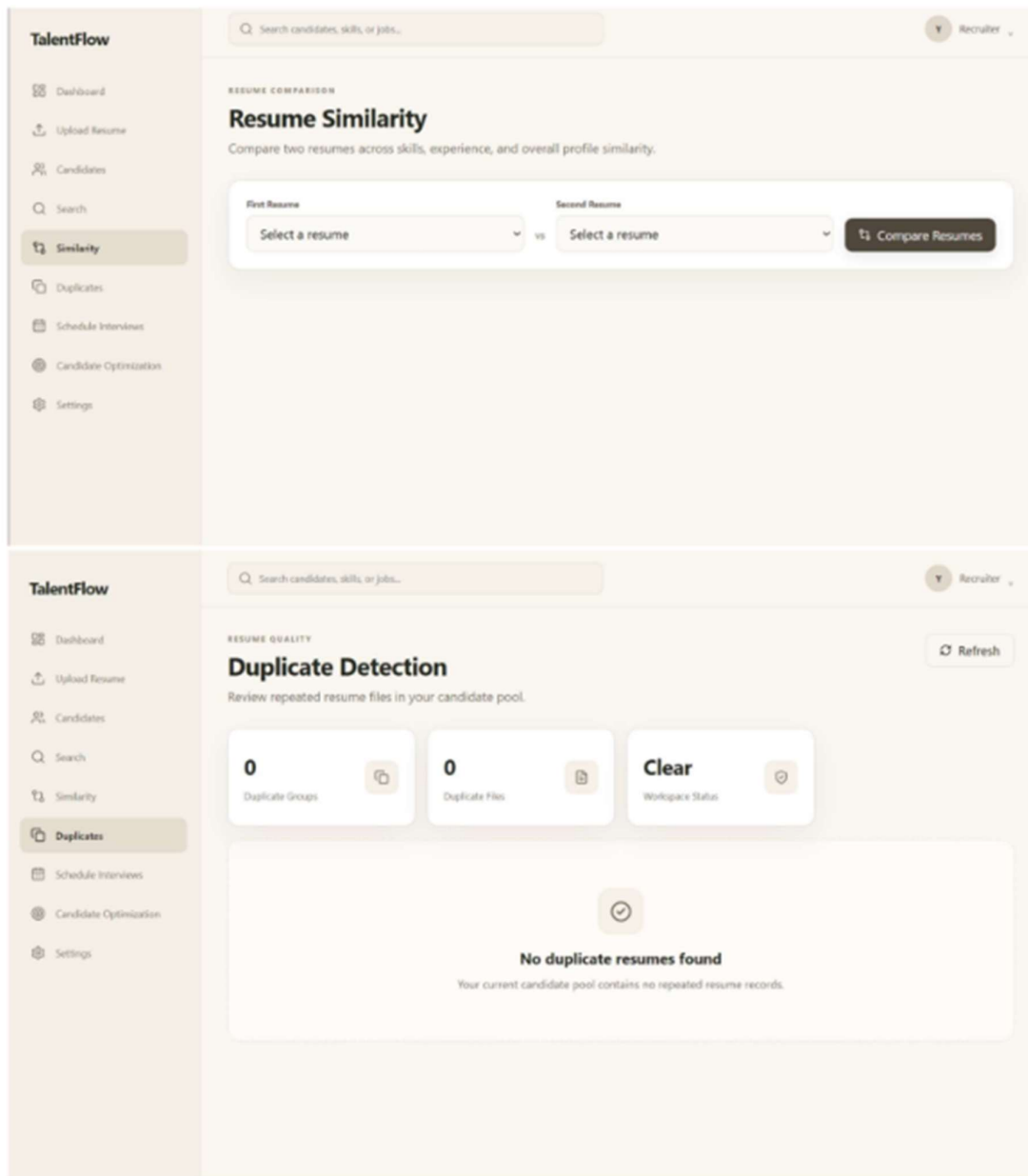
The project documentation identifies the recruiter-facing workflow as including application access, resume upload, candidate search, ranked search results, and candidate details. The final report should place the corresponding screenshots of the implemented application in this section.

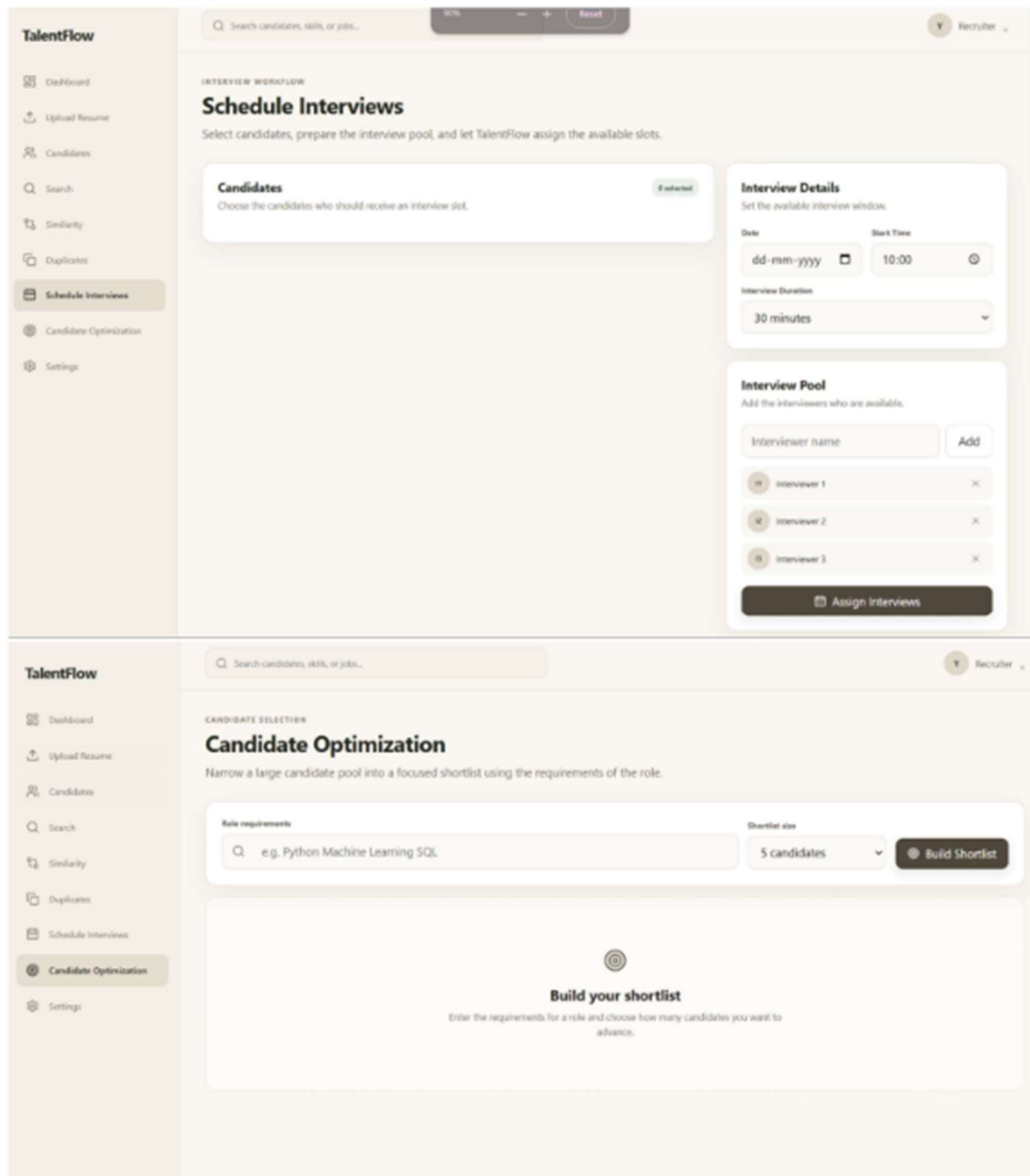
Screenshots should be numbered consistently, for example: Figure 6.1 Login Screen, Figure 6.2 Dashboard, Figure 6.3 Resume Upload, Figure 6.4 Candidate Search, Figure 6.5 Ranked Search Results, and Figure 6.6 Candidate Details. Actual screenshots should be inserted from the final implementation rather than recreated or assumed.











6.12 Performance Analysis

Performance analysis evaluates the computational behavior of the DSA components and the overall application workflow. The project objectives explicitly include analysis of the time and space complexity of the implemented algorithms.

The available project documentation provides theoretical complexity information for KMP and Edit Distance, but it does not provide a complete measured benchmark table for all algorithms. Therefore, execution time, memory consumption, throughput, and scalability values should be added only after running controlled tests on the final implementation.

For meaningful performance analysis, the test environment should be kept consistent and the input size should be varied where appropriate. Results should record the algorithm, input size, execution time, and relevant observations.

6.13 Expected System Results

The project documentation identifies the following expected outcomes from the integrated system:

Successful resume upload.

PDF text extraction.

Candidate object or structured candidate information creation.

Skill and keyword searching.

Spelling-tolerant searching.

Candidate matching.

Candidate ranking.

Ranked search results.

Candidate details.

6.14 Test Result Summary

Test Area	Expected Outcome
Resume Upload	PDF accepted and forwarded for processing
PDF Parsing	Text extracted from supported PDF
Candidate Processing	Candidate information created
Keyword Search	Relevant information identified
Edit Distance	Text variations handled according to implementation
Matching	Candidate relevance calculated
Ranking	Candidates ordered by implemented score
Frontend-Backend Integration	Request and response flow completed

6.15 Analysis

The documented architecture separates presentation, communication, application services, algorithmic processing, and data management. This separation supports modular testing because individual stages can be checked independently before the complete workflow is evaluated.

The use of DSA components gives the system a structured basis for text searching, comparison, matching, and optimization. At the same time, the effectiveness of the final system depends on the correctness of the implementation, the quality of extracted resume information, and the ranking or matching logic used by the application.

The available source material supports describing the functional workflow and theoretical algorithmic behavior. It does not support claiming specific overall accuracy, response-time benchmarks, or scalability measurements without additional execution evidence.

6.16 Summary

This chapter described the results and testing framework for TalentFlow. Functional testing covers resume upload, PDF text extraction, candidate processing, search, text comparison, matching, ranking, and candidate details. Algorithm testing covers the DSA components, while integration testing verifies communication between the frontend, REST APIs, backend, processing services, and result presentation.

The project documentation defines the expected system outcomes, but measured performance values and final pass/fail records must be obtained from actual execution of the completed implementation. The next chapter discusses challenges, solutions, and limitations of the system.

7. CHALLENGES AND LIMITATIONS

7.1 Issues Faced During Development

The development of TalentFlow involved integrating a recruiter-facing React frontend, a Java Spring Boot backend, REST-based communication, PDF resume processing, candidate data management, and several Data Structures and Algorithms. Because these components perform different responsibilities, one of the main development considerations was maintaining clear separation between presentation, application services, document processing, algorithmic processing, and candidate data management.

Resume processing also presented an important implementation consideration. TalentFlow accepts supported PDF resumes and uses Apache PDFBox to extract text before the extracted information is converted into a structured candidate representation. The quality of later search, similarity, matching, and ranking operations therefore depends on the information that can be obtained from the uploaded resume.

Another challenge was integrating DSA implementations with application-level functionality. The project contains algorithms for pattern searching, text comparison, similarity analysis, matching, optimization, and scalable resume processing. These components must be connected to the appropriate backend services so that they contribute to actual candidate-search and screening operations rather than remaining isolated algorithm demonstrations.

Frontend-backend integration was another important development concern. Recruiter actions such as resume upload, candidate search, similarity analysis, and scheduling are initiated in the React interface and communicated to the Spring Boot backend through REST APIs. Correct request and response handling is therefore required for the complete workflow to operate as intended.

Testing also requires separate verification of the major processing stages. The project documentation identifies resume upload, PDF text extraction, candidate processing, keyword search, text comparison, matching, ranking, and frontend-backend integration as test areas. The available project material does not provide a complete record of measured execution results for every test, so specific failure counts or performance figures are not stated here.

7.2 Solutions Applied

The system addresses the above development considerations through a layered architecture. The React frontend is kept responsible for recruiter interaction and presentation, while the Spring Boot backend coordinates application services, resume processing, candidate management, and algorithmic operations. REST APIs provide the communication boundary between the frontend and backend.

For resume processing, Apache PDFBox is used to extract text from supported PDF documents. The documented processing sequence is Resume PDF → PDFBox → Extracted Text → Candidate Object. This provides a structured input for subsequent candidate search, similarity, matching, and ranking operations.

The project also separates individual algorithmic functions into DSA components. Algorithms

such as Aho-Corasick, KMP, Rabin-Karp, Z-Function, Edit Distance, Suffix Array, Resume Similarity, Bipartite Matching, Candidate Optimization, and Scalable Resume Processing are included to support different processing requirements. This modular organization makes it possible to consider algorithmic operations separately from the user-interface layer.

The frontend-backend communication problem is addressed through defined REST API interfaces. A typical operation begins with a recruiter action in the React frontend, followed by an API request to the Spring Boot backend. The backend performs the required service or algorithmic operation and returns the resulting candidate or screening information to the frontend.

For testing and debugging, the project documentation recommends evaluating the individual stages before assessing the complete workflow. This modular testing approach is consistent with the architecture because upload, parsing, candidate processing, search, comparison, matching, ranking, and integration can be checked as separate functional areas before complete system evaluation.

7.3 Limitations of the System

The current system has limitations that should be considered when interpreting its results. First, the resume-processing workflow is based on supported PDF documents and extracted textual information. The quality and completeness of candidate information therefore depend on the structure and content of the uploaded resume and on the information that can be extracted from it.

Second, the project documentation defines search, similarity, matching, and ranking functionality, but the available source material does not provide sufficient execution evidence to claim a specific overall accuracy percentage, response-time benchmark, or scalability measurement. Such values should be added only after controlled testing of the completed implementation.

Third, candidate ranking and matching depend on the implemented scoring and comparison logic. A ranking produced by the system should therefore be treated as decision-support information rather than as a replacement for recruiter evaluation. Recruiters may still need to examine the original candidate information and apply job-specific judgement.

Fourth, the system is designed around the technologies and modules described in the project, including React, Spring Boot, REST APIs, Apache PDFBox, and the implemented DSA components. Expanding the system to substantially larger resume collections or more advanced recruitment workflows may require additional optimization, storage design, validation, and performance testing.

Finally, the available project documentation describes the intended functional workflow and expected outcomes, but it does not establish comprehensive production-scale validation. Further testing with larger and more diverse resume collections would be required before making claims about large-scale deployment performance.

7.4 Summary

This chapter discussed the main development considerations and limitations of TalentFlow. The project requires coordination between the React frontend, REST API layer, Spring Boot backend, PDF processing, candidate data management, and DSA components. The layered and modular architecture provides a structured approach for integrating these functions. At the same time, resume extraction quality, ranking logic, supported input formats, and the absence of complete measured benchmark results represent important limitations. These limitations provide a basis for the improvements discussed in the next chapter.

8. CONCLUSION AND FUTURE WORK

8.1 Conclusion

TalentFlow – Resume Search and Candidate Screening System demonstrates how Data Structures and Algorithms can be integrated into a practical recruitment application. The project addresses the difficulty of manually examining large numbers of resumes by providing a structured workflow for resume uploading, PDF processing, candidate information management, searching, similarity analysis, candidate matching, ranking, and interview scheduling.

The implemented architecture combines a React-based frontend with a Java Spring Boot backend through REST APIs. Apache PDFBox supports text extraction from supported PDF resumes, after which candidate information can be processed by the application and algorithmic layers. This separation provides a clear workflow from recruiter interaction to resume processing, DSA-based operations, and presentation of candidate-related results.

A major outcome of the project is the practical application of multiple DSA techniques to recruitment-related text and matching problems. The project incorporates Aho-Corasick, KMP, Rabin-Karp, Z-Function, Edit Distance, Suffix Array, Resume Similarity, Bipartite Matching, Candidate Optimization, and Scalable Resume Processing. These techniques provide algorithmic support for searching, comparison, similarity analysis, matching, optimization, and candidate screening.

The system therefore serves both as a recruitment-support application and as a practical demonstration of applying DSA concepts to a real-world software problem. The available project documentation supports the expected functional workflow and the integration of the major components. However, complete measured accuracy, response-time, and scalability values should only be reported after obtaining corresponding execution and benchmark evidence.

8.2 Future Work

The current TalentFlow system provides a foundation that can be extended in several directions. Future improvements can focus on increasing the robustness of resume processing, improving candidate analysis, expanding scalability, and strengthening the overall recruitment workflow.

Advanced Resume Parsing: Resume extraction can be extended to handle a wider variety of resume layouts and document structures, improving the consistency of structured candidate information.

Improved Candidate Similarity: More advanced similarity techniques can be incorporated to compare skills, education, experience, projects, and other candidate attributes more effectively.

Enhanced Candidate Ranking: The ranking component can be extended with clearly defined and configurable scoring criteria so that recruiters can adjust the importance of different job requirements.

Scalable Processing: The resume-processing and search components can be optimized and evaluated with larger candidate collections to establish measurable scalability and performance

characteristics.

Expanded Search Capabilities: Future versions can support more flexible search criteria, combinations of multiple requirements, and richer filtering of candidate information.

Improved Duplicate Analysis: Duplicate and closely related candidate records can be analyzed using well-defined similarity criteria and validated thresholds.

Recruitment Workflow Expansion: The existing screening workflow can be extended with additional recruitment-management features around candidate review and interview processing.

Comprehensive Evaluation: Future work should include controlled experiments with representative resume datasets and record execution time, input size, correctness, and other relevant performance measures.

Deployment and Reliability Improvements: The application can be further evaluated and optimized for larger-scale deployment, reliability, maintainability, and operational monitoring.

8.3 Skills Learned During Development

The development of TalentFlow provided practical experience across Data Structures and Algorithms, full-stack application development, document processing, API communication, testing, and software integration.

Data Structures and Algorithms: Practical understanding of string matching, dynamic programming, text indexing, similarity analysis, graph-based matching, optimization, and complexity analysis.

Java and Spring Boot: Experience in implementing backend services, application logic, resume-processing functions, and REST API functionality.

React Frontend Development: Experience in building recruiter-facing interfaces for resume upload, candidate search, candidate information access, and screening-related result presentation.

REST API Integration: Understanding of communication between frontend and backend components through defined API interfaces.

PDF Processing: Practical use of Apache PDFBox for extracting text from supported PDF resumes and passing extracted information to later processing stages.

Software Architecture: Understanding of layered architecture and separation of presentation, communication, application services, algorithmic processing, and data management.

Testing and Debugging: Experience in breaking the application workflow into testable stages and checking resume processing, search, comparison, matching, ranking, and integration behavior.

Problem Solving and Integration: Experience in connecting algorithmic solutions with application modules so that DSA techniques address a practical recruitment problem.

8.4 Summary

TalentFlow demonstrates the integration of DSA-based processing with a full-stack resume search and candidate screening application. The project combines resume processing, structured candidate information, algorithmic search and comparison, matching and ranking, and recruiter-oriented application functionality. The work establishes a foundation for further improvements in parsing, similarity analysis, ranking, scalability, evaluation, and recruitment workflow support. The technical experience gained during development also provides practical exposure to DSA, Java, Spring Boot, React, PDF processing, REST APIs, testing, and system integration.

REFERENCES

The following references support the technologies, algorithms, and implementation concepts described in the TalentFlow Resume Search and Candidate Screening System. The project documentation identifies books, tutorials, APIs, and documentation sites as the sources category for the final References section

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., Introduction to Algorithms, 3rd ed., MIT Press, 2009.
2. Knuth, D. E., Morris, J. H., and Pratt, V. R., “Fast Pattern Matching in Strings,” SIAM Journal on Computing, Vol. 6, No. 2, 1977, pp. 323–350.
3. Aho, A. V. and Corasick, M. J., “Efficient String Matching: An Aid to Bibliographic Search,” Communications of the ACM, Vol. 18, No. 6, 1975, pp. 333–340.
4. Apache Software Foundation, Apache PDFBox Documentation and Project Resources. Used for PDF document reading and text extraction in the resume-processing workflow.
5. Oracle, Java Documentation and Java Platform, Standard Edition Documentation. Used as the reference for Java development and APIs.
6. Spring, Spring Boot Reference Documentation. Used as the reference for the Java Spring Boot backend and web application development.
7. React, React Documentation and API Reference. Used as the reference for the recruiter-facing React frontend.
8. Knuth, D. E., The Art of Computer Programming, Volume 3: Sorting and Searching, Addison-Wesley.
9. Levenshtein, V. I., “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” Soviet Physics Doklady, Vol. 10, 1966, pp. 707–710.
10. Apache Software Foundation, Apache PDFBox API and User Documentation. Used for the documented Resume PDF → PDFBox → Extracted Text → Candidate Object processing sequence.
11. Project Documentation, TalentFlow – Resume Search and Candidate Screening System. Internal project documentation covering system requirements, design, DSA algorithms, implementation, testing, challenges, limitations, conclusion, and future work.

Note: The project files explicitly establish the use of React, Java Spring Boot, REST APIs, Apache PDFBox, and the listed DSA components. The project documentation also identifies Aho-Corasick, KMP, Rabin-Karp, Z-Function, Edit Distance, Suffix Array, Resume Similarity, Bipartite Matching, Candidate Optimization, and Scalable Resume Processing.